

UNIVERSIDAD AUTÓNOMA GABRIEL RENE MORENO
*Facultad de Ingeniería en Ciencias de la Computación y
Telecomunicaciones*



“CSS Shapes + Transforms”

Materia : TECNOLOGIA WEB - SA

Carrera : Ingeniería En Sistemas

Docente : **Evans Balcazar Veizaga**

Estudiantes :

Arnez Fernández Fabio Alejandro 221043721

Loaiza Labarden Reymar 216027438

Marca Peñaranda Carlos Diego 221090436

Semestre : 1 / 2026

Santa Cruz – Bolivia

	FORMULARIO DE PROYECTOS FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIÓN				
CARRERA:	Ingeniería en Sistemas				
UNIDAD/MATERIA:	Tecnología Web INF-513 SA				
TITULO:	CSS Shapes + Transforms				
MODALIDAD: (Expo . /Concurso)	Presencial				
RESPONSABLE/DOCENTE:	ING. EVANS BALCÁZAR VEIZAGA				
INTEGRANTES		REGISTRO/COD. DOC			
Arnez Fernández Fabio Alejandro		221043721			
Loaiza Labarden Reymar		216027438			
Marca Peñaranda Carlos Diego		221090436			
GESTION:	1/2026	CD:	NO		
TRABAJO: INVESTIGACION ☐		FORMATIVA: ☒			
LINEA DE INVESTIGACION	Tecnologías Web orientadas al diseño y desarrollo de interfaces interactivas, accesibles y visualmente atractivas mediante el uso de CSS Shapes y CSS Transforms.				
DESCRIPCION	Esta investigación tiene como propósito aprender y aplicar técnicas modernas de diseño web mediante CSS Shapes y CSS Transforms, permitiendo crear interfaces más dinámicas, atractivas y funcionales para mejorar la experiencia del usuario.				
FECHA DE PRESENTACION	03 / 06 / 2026				

Contenido

1.	INTRODUCCION	1
2.	ANTECEDENTES Y JUSTIFICACION	1
2.1.	ANTECEDENTES	1
2.2.	JUSTIFICACION	2
3.	EL PROBLEMA	3
3.1.	SITUACION PROBLEMÁTICA	3
3.2.	FORMULACION DEL PROBLEMA.....	3
4.	OBJETIVOS	4
4.1.	OBJETIVO GENERAL	4
4.2.	OBJETIVO ESPECIFICO	4
5.	METODOLOGIA	4
5.1.	Fase de Investigación y Análisis.....	4
5.1.1.	<i>Técnicas utilizadas:</i>	5
5.1.2.	<i>Recursos utilizados:</i>	5
5.2.	Fase de Diseño	5
5.2.1.	<i>Técnicas utilizadas:</i>	5
5.2.2.	<i>Herramientas utilizadas:</i>	5
5.3.	Fase de Implementación.....	5
5.3.1.	<i>Técnicas utilizadas:</i>	6
5.3.2.	<i>Recursos utilizados:</i>	6
5.4.	Fase de Pruebas y Validación	6
5.4.1.	<i>Técnicas utilizadas:</i>	6
5.4.2.	<i>Resultados esperados:</i>	6
6.	MARCO TEORICO	7
6.1.	Desarrollo Web Moderno	7
6.2.	¿Qué es CSS Shapes?	7
6.2.1.	<i>Propiedades principales de CSS Shapes</i>	7
6.3.	¿Qué es CSS Transforms?	8
6.3.1.	<i>Propiedades principales de CSS Transforms</i>	9
6.4.	Aplicación de Shapes y Transforms en el Desarrollo Web	10
7.	DESARROLLO	10
7.1.	Planificación del Proyecto.....	11
7.2.	Diseño de la Interfaz Web.....	11
7.3.	Desarrollo del Módulo CSS Transform.....	12
7.3.1.	<i>Estructura HTML</i>	12
7.3.2.	<i>Implementación de translate()</i>	13
7.3.3.	<i>Implementación de rotate()</i>	13
7.3.4.	<i>Implementación de scale()</i>	13
7.3.5.	<i>Implementación de skew()</i>	14

7.3.6.	<i>Implementación de matrix()</i>	14
7.3.7.	<i>Transformaciones 3D</i>	14
7.3.8.	<i>Combinación de Transformaciones</i>	14
7.4.	Desarrollo del Módulo CSS Shapes	15
7.4.1.	<i>Implementación de circle()</i>	15
7.4.2.	<i>Implementación de ellipse()</i>	15
7.4.3.	<i>Implementación de polygon()</i>	15
7.4.4.	<i>Implementación de inset()</i>	16
7.4.5.	<i>Implementación de url()</i>	16
7.5.	Implementación del Diseño Responsivo	16
7.6.	Uso de Variables CSS	17
7.7.	Implementación de Efectos Visuales	17
7.8.	Pruebas y Validaciones	17
7.9.	Resultado del Desarrollo	18
8.	RESULTADOS	18
9.	CONCLUSIONES	19
10.	RECOMENDACIONES	20
11.	BIBLIOGRAFIA CITADA	20
12.	ANEXOS	20
12.1.	Capturas de la interfaz del proyecto	20
12.2.	Estructura del Proyecto	22

1. INTRODUCCION

En la actualidad, el desarrollo web moderno busca crear interfaces más atractivas, dinámicas e interactivas para mejorar la experiencia de los usuarios. Gracias a la evolución de CSS3, existen herramientas avanzadas que permiten diseñar elementos visuales innovadores sin depender completamente de imágenes o librerías externas. Entre estas herramientas destacan CSS Shapes y CSS Transforms, tecnologías utilizadas para crear formas personalizadas y aplicar transformaciones visuales a los elementos de una página web.

Sin embargo, muchas personas que recién comienzan en el desarrollo web poseen únicamente conocimientos básicos de HTML y CSS, por lo que desconocen las funcionalidades y ventajas que ofrecen CSS Shapes y CSS Transforms. Esta falta de conocimiento limita la creación de diseños modernos e interactivos, provocando que las interfaces web sean simples, poco dinámicas y visualmente menos atractivas.

Por esta razón, el presente proyecto tiene como finalidad explicar el funcionamiento y las características principales de CSS Shapes y CSS Transforms mediante ejemplos sencillos y prácticos que permitan comprender cómo estas herramientas pueden mejorar el diseño visual de una página web. Asimismo, se busca demostrar cómo es posible modificar la forma, posición, tamaño y orientación de los elementos HTML utilizando únicamente propiedades CSS.

Para el desarrollo del proyecto se empleará una metodología práctica basada en la investigación de documentación web, análisis de propiedades CSS y elaboración de ejemplos interactivos utilizando HTML y CSS. A través de demostraciones visuales se presentará el comportamiento de las distintas transformaciones y formas que pueden aplicarse dentro de una interfaz web.

Como resultado esperado, se pretende que los estudiantes y usuarios comprendan la utilidad de CSS Shapes y CSS Transforms, reconozcan sus ventajas dentro del desarrollo frontend y puedan aplicar estas herramientas en la creación de páginas web más modernas, creativas y visualmente atractivas.

2. ANTECEDENTES Y JUSTIFICACION

2.1. ANTECEDENTES

El desarrollo web ha evolucionado constantemente con la incorporación de nuevas tecnologías enfocadas en mejorar la apariencia visual y la interacción de las páginas web. Con la llegada de CSS3 surgieron diversas herramientas que permiten crear diseños más modernos y dinámicos sin necesidad de utilizar imágenes complejas o grandes cantidades de código JavaScript.

Entre estas herramientas destacan CSS Shapes y CSS Transforms. CSS Shapes permite modificar la forma en que el contenido se adapta alrededor de figuras geométricas personalizadas, mientras que CSS Transforms ofrece la posibilidad de rotar, mover, escalar e inclinar elementos HTML dentro de una interfaz web. Estas propiedades son ampliamente utilizadas en sitios modernos para mejorar la presentación visual y brindar una experiencia más atractiva al usuario.

A pesar de la existencia de estas herramientas, muchos estudiantes y desarrolladores principiantes poseen conocimientos limitados sobre su funcionamiento y aplicación práctica. Generalmente se enfocan únicamente en estilos básicos de CSS, dejando de lado funcionalidades avanzadas que pueden mejorar significativamente el diseño frontend de una página web.

Por ello, surge la necesidad de investigar y explicar las características principales de CSS Shapes y CSS Transforms mediante ejemplos sencillos que faciliten su comprensión y aplicación dentro del desarrollo web.

2.2. JUSTIFICACION

El presente proyecto se realiza con la finalidad de dar a conocer las funcionalidades y ventajas que ofrecen CSS Shapes y CSS Transforms dentro del diseño web moderno. La investigación es importante porque permitirá que estudiantes y desarrolladores comprendan cómo utilizar estas herramientas para crear interfaces más dinámicas, creativas y visualmente atractivas.

Desde el punto de vista teórico, el proyecto aportará conocimientos sobre las propiedades y características principales de CSS Shapes y CSS Transforms, explicando su funcionamiento y comportamiento dentro de una página web.

En el aspecto metodológico, se empleará una metodología práctica basada en ejemplos interactivos desarrollados con HTML y CSS, permitiendo observar visualmente cómo actúan las diferentes transformaciones y formas aplicadas a los elementos web.

Finalmente, en el aspecto práctico, el proyecto contribuirá al fortalecimiento de habilidades en desarrollo frontend, ayudando a mejorar la calidad visual de las interfaces web mediante el uso de herramientas modernas de CSS3. Además, permitirá que los usuarios comprendan que es posible crear efectos visuales avanzados utilizando únicamente CSS, reduciendo la necesidad de recursos externos y optimizando el diseño de las páginas web.

3. EL PROBLEMA

3.1. SITUACION PROBLEMÁTICA

En la actualidad, el desarrollo web moderno enfrenta diversos desafíos relacionados con la creación de interfaces visualmente atractivas, dinámicas e innovadoras. Aunque CSS3 ofrece herramientas avanzadas como CSS Shapes y CSS Transforms, muchos estudiantes y desarrolladores web aún desconocen las funcionalidades y ventajas que estas tecnologías proporcionan dentro del diseño frontend. Esta situación provoca que numerosas páginas web mantengan diseños básicos y poco interactivos, limitando la creatividad y la experiencia visual del usuario.

El diagnóstico actual evidencia que gran parte de los desarrolladores principiantes continúa utilizando únicamente propiedades tradicionales de CSS, enfocándose en estructuras rectangulares simples y efectos visuales limitados. Como consecuencia, muchas interfaces web presentan poca innovación visual y desaprovechan herramientas modernas capaces de mejorar significativamente la presentación y la interacción de los elementos dentro de una página web.

3.2. FORMULACION DEL PROBLEMA

Las principales causas de esta problemática son las siguientes:

- **Falta de conocimiento:** Muchos estudiantes y desarrolladores no conocen las propiedades y funcionalidades que ofrecen CSS Shapes y CSS Transforms, dificultando su aplicación en proyectos web.
- **Escasez de ejemplos prácticos:** Existen pocos materiales explicativos simples y prácticos que permitan comprender fácilmente el uso de estas herramientas dentro del desarrollo frontend.
- **Uso limitado de CSS avanzado:** Generalmente se utilizan únicamente estilos básicos de CSS, dejando de lado técnicas modernas que permiten crear interfaces más creativas e interactivas.
- **Dependencia de herramientas externas:** En muchos casos se utilizan librerías o recursos adicionales para realizar efectos visuales que podrían implementarse directamente mediante CSS.

Si esta situación continúa, los desarrolladores seguirán creando páginas web con diseños limitados y menos atractivos visualmente, desaprovechando el potencial de CSS3 para generar experiencias modernas e interactivas. Además, la falta de conocimiento sobre estas tecnologías reducirá las posibilidades de innovación dentro del diseño web frontend.

4. OBJETIVOS

4.1. OBJETIVO GENERAL

Analizar y demostrar las funcionalidades de CSS Shapes y CSS Transforms mediante ejemplos prácticos para mejorar el conocimiento y la aplicación de herramientas avanzadas de CSS3 en el desarrollo de interfaces web modernas e interactivas.

4.2. OBJETIVO ESPECIFICO

- Identificar las principales propiedades y características de CSS Shapes y CSS Transforms utilizadas en el desarrollo web.
- Explicar el funcionamiento de las herramientas CSS Shapes y CSS Transforms mediante ejemplos visuales y prácticos.
- Diseñar ejemplos interactivos utilizando HTML y CSS que permitan demostrar la aplicación de Shapes y Transforms dentro de una página web.
- Evaluar cómo el uso de CSS Shapes y CSS Transforms contribuye a mejorar la apariencia visual y la interacción de las interfaces web.
- Demostrar que es posible crear efectos visuales modernos utilizando únicamente propiedades avanzadas de CSS3.

5. METODOLOGIA

Para el desarrollo del presente proyecto se empleará una metodología práctica y descriptiva orientada al análisis y demostración de las funcionalidades de CSS Shapes y CSS Transforms dentro del desarrollo web moderno. La metodología aplicada permitirá alcanzar los objetivos planteados mediante la investigación teórica y la implementación de ejemplos prácticos desarrollados con tecnologías web.

Asimismo, se utilizará una metodología de desarrollo basada en el Proceso Unificado de Desarrollo de Software (RUP), debido a que permite organizar de manera estructurada las actividades necesarias para el análisis, diseño, implementación y validación del proyecto.

La metodología se desarrollará en las siguientes etapas:

5.1. Fase de Investigación y Análisis

En esta etapa se realizará la recopilación de información relacionada con CSS3, especialmente sobre las propiedades CSS Shapes y CSS Transforms. Se investigarán conceptos, funcionalidades, sintaxis, aplicaciones y ejemplos de uso mediante documentación oficial, tutoriales y recursos especializados en desarrollo web.

Además, se analizarán las ventajas que estas herramientas ofrecen para mejorar el diseño visual y la interacción de las interfaces web modernas.

5.1.1. Técnicas utilizadas:

- Investigación documental.
- Revisión bibliográfica digital.
- Análisis de documentación técnica.

5.1.2. Recursos utilizados:

- Navegador web.
- Documentación oficial de CSS.
- Material educativo y ejemplos de desarrollo frontend.

5.2. Fase de Diseño

Durante esta fase se planificará la estructura de los ejemplos prácticos que serán utilizados para demostrar el funcionamiento de CSS Shapes y CSS Transforms. Se definirán los elementos visuales, la distribución del contenido y los efectos de transformación que serán aplicados dentro de las páginas web.

Asimismo, se realizará el diseño básico de la interfaz utilizando HTML y CSS.

5.2.1. Técnicas utilizadas:

- Diseño estructural de interfaces web.
- Organización de componentes visuales.
- Elaboración de prototipos básicos.

5.2.2. Herramientas utilizadas:

- HTML5.
- CSS3.
- Visual Studio Code.

5.3. Fase de Implementación

En esta etapa se desarrollarán ejemplos prácticos e interactivos utilizando propiedades de CSS Shapes y CSS Transforms. Se aplicarán transformaciones como rotación, escalado, desplazamiento e inclinación de elementos HTML, además de la creación de formas personalizadas utilizando CSS Shapes.

El desarrollo permitirá demostrar de manera visual cómo estas herramientas pueden mejorar la apariencia y dinamismo de una página web.

5.3.1. Técnicas utilizadas:

- Programación frontend.
- Implementación de estilos avanzados con CSS3.
- Desarrollo de ejemplos interactivos.

5.3.2. Recursos utilizados:

- Computadora.
- Editor de código Visual Studio Code.
- Navegadores web para pruebas.

5.4. Fase de Pruebas y Validación

Finalmente, se realizarán pruebas visuales y funcionales para verificar el correcto comportamiento de las propiedades CSS implementadas. Se evaluará la compatibilidad de los ejemplos desarrollados en distintos navegadores web y se comprobará que los resultados obtenidos cumplan con los objetivos planteados en el proyecto.

5.4.1. Técnicas utilizadas:

- Pruebas funcionales.
- Validación visual de interfaces.
- Verificación de compatibilidad entre navegadores.

5.4.2. Resultados esperados:

- Comprensión del funcionamiento de CSS Shapes y CSS Transforms.
- Desarrollo de interfaces más dinámicas y atractivas.
- Aplicación práctica de herramientas avanzadas de CSS3 en páginas web.

6. MARCO TEORICO

6.1. Desarrollo Web Moderno

El desarrollo web moderno se enfoca en la creación de interfaces dinámicas, interactivas y visualmente atractivas que mejoren la experiencia del usuario. Gracias a la evolución de tecnologías frontend como HTML5 y CSS3, actualmente es posible implementar efectos visuales avanzados, animaciones y diseños personalizados sin necesidad de depender completamente de imágenes o librerías externas.

Dentro de este contexto, CSS3 incorpora múltiples herramientas orientadas a mejorar la presentación visual de las páginas web. Entre ellas destacan CSS Shapes y CSS Transforms, tecnologías que permiten crear diseños más creativos y modernos mediante formas geométricas y transformaciones visuales aplicadas a los elementos HTML.

Estas herramientas han ganado importancia dentro del diseño frontend debido a que permiten desarrollar interfaces más innovadoras, mejorar la interacción del usuario y optimizar la apariencia visual de los sitios web modernos.

6.2. ¿Qué es CSS Shapes?

CSS Shapes es una especificación de CSS3 que permite definir formas geométricas no rectangulares alrededor de las cuales puede fluir el contenido de una página web. Esta tecnología rompe con el modelo tradicional de cajas rectangulares utilizado en CSS, permitiendo crear diseños más creativos y visualmente atractivos.

Mediante CSS Shapes es posible hacer que textos e imágenes se adapten alrededor de figuras como círculos, elipses, polígonos y otras formas personalizadas. Esto facilita la creación de diseños editoriales modernos y layouts más dinámicos.

Las CSS Shapes permiten desarrollar interfaces más naturales y mejorar significativamente la presentación visual de los contenidos web.

6.2.1. Propiedades principales de CSS Shapes

shape-outside

La propiedad shape-outside define la forma alrededor de la cual fluirá el contenido adyacente. Esta propiedad admite diferentes tipos de formas:

- **circle():** Permite crear formas circulares.
- **ellipse():** Genera formas elípticas.
- **polygon():** Permite crear polígonos personalizados.
- **inset():** Define formas rectangulares internas.

- **URL de imágenes:** Utiliza imágenes para definir formas mediante transparencia.

shape-margin

Define el espacio o margen entre la forma y el contenido que la rodea, mejorando la separación visual de los elementos.

shape-image-threshold

Permite especificar el nivel de transparencia utilizado para extraer formas a partir de imágenes.

6.2.1.1. Ventajas de CSS Shapes

- Permite crear diseños más creativos y modernos.
- Mejora la presentación visual del contenido.
- Facilita la creación de estilos editoriales dinámicos.
- Reduce el uso de imágenes decorativas complejas.
- Funciona únicamente mediante CSS, sin necesidad de JavaScript.

6.2.1.2. Limitaciones de CSS Shapes

- Compatibilidad limitada en navegadores antiguos.
- Mayor complejidad al diseñar formas avanzadas.
- Solo afecta el flujo de ciertos elementos HTML.

6.3. ¿Qué es CSS Transforms?

CSS Transforms es una especificación de CSS3 que permite aplicar transformaciones geométricas a elementos HTML sin alterar el flujo normal del documento. Estas transformaciones pueden realizarse en dos dimensiones (2D) y tres dimensiones (3D).

Las transformaciones CSS son ampliamente utilizadas para crear efectos visuales modernos, animaciones suaves y elementos interactivos dentro de interfaces web. Gracias a estas propiedades es posible modificar la posición, tamaño, orientación e inclinación de elementos de forma dinámica.

CSS Transforms mejora considerablemente la experiencia visual del usuario y permite desarrollar interfaces más atractivas sin utilizar código JavaScript adicional.

6.3.1. Propiedades principales de CSS Transforms

`transform`

Es la propiedad principal utilizada para aplicar transformaciones a elementos HTML.

Las funciones más utilizadas son:

- `translate()`: Desplaza elementos horizontal o verticalmente.
- `rotate()`: Permite rotar elementos.
- `scale()`: Modifica el tamaño de los elementos.
- `skew()`: Inclina elementos creando efectos visuales.
- `matrix()`: Permite aplicar transformaciones complejas mediante matrices matemáticas.

`transform-origin`

Define el punto desde el cual se aplicarán las transformaciones.

`transform-style`

Permite controlar cómo se visualizarán los elementos hijos dentro de espacios tridimensionales.

`perspective`

Define la distancia entre el observador y el plano $Z=0$, creando la sensación de profundidad 3D. Se aplica al elemento padre. Valores pequeños producen una perspectiva más pronunciada; valores grandes la suavizan.

`perspective-origin`

Establece el punto de fuga de la perspectiva dentro del elemento padre; equivale a mover la cámara lateralmente. Valor por defecto: 50% 50% (centro).

`backface-visibility`

Controla si la cara trasera de un elemento 3D es visible cuando apunta hacia el usuario. Con el valor `hidden` la cara posterior se oculta, lo que es esencial para efectos de tarjeta volteada (card flip).

6.3.1.1. Ventajas de CSS Transforms

- Permite crear efectos visuales dinámicos.

- Mejora la interacción y experiencia del usuario.
- Posee aceleración por hardware en navegadores modernos.
- Facilita la creación de animaciones fluidas.
- Compatible con la mayoría de navegadores actuales.

6.3.1.2. Limitaciones de CSS Transforms

- Las transformaciones no modifican el espacio real ocupado en el layout.
- Las transformaciones 3D pueden resultar complejas para desarrolladores principiantes.
- Algunas animaciones complejas pueden afectar el rendimiento en dispositivos de bajos recursos.

6.4. Aplicación de Shapes y Transforms en el Desarrollo Web

Actualmente, CSS Shapes y CSS Transforms son ampliamente utilizados en el desarrollo de interfaces web modernas debido a su capacidad para mejorar la apariencia visual y la interacción de los sitios web. Estas tecnologías permiten crear diseños innovadores, efectos dinámicos y experiencias más atractivas para los usuarios.

Muchos sitios web modernos utilizan Shapes y Transforms para implementar tarjetas animadas, galerías interactivas, efectos hover, diseños editoriales y elementos visuales personalizados. Además, estas herramientas ayudan a reducir la dependencia de imágenes y librerías externas, optimizando el rendimiento y la flexibilidad del diseño frontend.

Por ello, el presente proyecto busca explicar y demostrar mediante ejemplos prácticos el funcionamiento y aplicación de CSS Shapes y CSS Transforms, permitiendo comprender cómo estas tecnologías pueden utilizarse para desarrollar páginas web más modernas, creativas e interactivas.

7. DESARROLLO

El desarrollo del proyecto se realizó aplicando una metodología incremental y práctica orientada al diseño y construcción de interfaces web educativas utilizando HTML5 y CSS3. El objetivo principal fue desarrollar una guía interactiva sobre propiedades avanzadas de CSS, específicamente CSS Transform y CSS Shapes, permitiendo mostrar ejemplos visuales, sintaxis y demostraciones dinámicas para facilitar el aprendizaje de estas tecnologías.

Durante el proceso de desarrollo se trabajó en distintas etapas: análisis del contenido, diseño de la interfaz, estructuración HTML, implementación de estilos CSS,

incorporación de animaciones y transformaciones, pruebas responsivas y optimización visual.

7.1. Planificación del Proyecto

En la primera etapa se definió el alcance del proyecto y los contenidos que serían implementados dentro de la guía web interactiva.

Se decidió dividir el sistema en dos módulos principales:

- Módulo de Transformaciones CSS
- Módulo de CSS Shapes (shape-outside)

Cada módulo tendría:

- Explicación teórica
- Sintaxis
- Ejemplo de código
- Demostración visual interactiva
- Diseño moderno y responsivo

También se estableció una estructura visual uniforme para mantener coherencia entre todas las secciones del proyecto.

7.2. Diseño de la Interfaz Web

Antes de iniciar la codificación se realizó el diseño conceptual de la interfaz.

Se optó por un diseño:

- Moderno
- Minimalista
- Responsivo
- Interactivo
- Educativo

Para ello se utilizaron:

- Paletas de colores oscuras
- Gradientes
- Tarjetas informativas

- Sombras y efectos visuales
- Tipografías modernas obtenidas desde Google Fonts

El diseño fue organizado mediante tarjetas (cards) que contienen:

- Encabezado
- Código de ejemplo
- Área demostrativa
- Explicación técnica

Esta estructura permitió que el usuario pueda comprender cada propiedad CSS de manera visual y práctica.

7.3. Desarrollo del Módulo CSS Transform

En este módulo se desarrolló una guía completa sobre las transformaciones 2D y 3D de CSS.

El archivo principal utilizado fue:

- index.html
- style.css
- app.js

7.3.1. Estructura HTML

Se construyó una estructura semántica utilizando etiquetas HTML5:

```
<header>
<main>
<section>
<footer>
```

Cada propiedad de transformación fue organizada dentro de una tarjeta independiente utilizando:

```
<section class="method-card">
```

Dentro de cada tarjeta se añadieron:

- Nombre del método
- Descripción

- Sintaxis
- Código demostrativo
- Área interactiva

7.3.2. Implementación de *translate()*

Se desarrolló una demostración visual de la propiedad:

```
transform: translate(100px);
```

Esta transformación permite mover un elemento horizontal o verticalmente sin alterar el flujo del documento.

La demostración se implementó utilizando eventos hover:

```
.box-1:hover {
  transform: translate(100px);
}
```

El usuario puede observar visualmente cómo el elemento cambia de posición al interactuar con él.

7.3.3. Implementación de *rotate()*

Para explicar las rotaciones se implementó:

```
transform: rotate(45deg);
```

La propiedad permite girar elementos sobre su eje central.

Se añadió una animación visual que rota la figura al pasar el cursor:

```
.box-2:hover {
  transform: rotate(45deg);
}
```

También se explicaron variantes como:

- rotateX()
- rotateY()
- rotateZ()

7.3.4. Implementación de *scale()*

La propiedad *scale()* fue utilizada para demostrar cambios de tamaño dinámicos.

```
transform: scale(2);
```

Se implementó un efecto de ampliación:

```
.box-3:hover {
  transform: scale(2);
}
```

Esto permitió visualizar el comportamiento del escalado tanto horizontal como vertical.

7.3.5. Implementación de skew()

La propiedad skew() se desarrolló para demostrar distorsiones angulares.

```
transform: skew(-45deg);
```

El efecto fue aplicado mediante:

```
.box-4:hover {
  transform: skew(-45deg);
}
```

Esta funcionalidad permite inclinar elementos sobre los ejes X y Y.

7.3.6. Implementación de matrix()

Se desarrolló un ejemplo avanzado utilizando matrices de transformación:

```
transform: matrix(2, 0.45, 0.45, 2, 20, 20);
```

Esta propiedad combina:

- Escalado
- Sesgado
- Traslación

El objetivo fue mostrar una transformación compuesta en una sola línea CSS.

7.3.7. Transformaciones 3D

Se implementaron efectos tridimensionales mediante:

```
transform: rotateX(180deg);
```

Además, se utilizó:

```
perspective: 1000px;
```

Esto permitió crear profundidad visual y efectos de volteo 3D.

7.3.8. Combinación de Transformaciones

Se desarrolló una sección especializada para mostrar múltiples transformaciones simultáneas:

```
transform: translate(50px, 30px) rotate(15deg) scale(1.2);
```

El objetivo fue demostrar que el orden de aplicación modifica el resultado final de la transformación.

7.4. Desarrollo del Módulo CSS Shapes

En este módulo se desarrolló una guía visual enfocada en la propiedad shape-outside.

Los archivos utilizados fueron:

- `shape_index.html`
- `shape_outside.css`

El objetivo principal fue demostrar cómo el texto puede adaptarse alrededor de formas geométricas personalizadas.

7.4.1. Implementación de *circle()*

Se desarrolló una forma circular utilizando:

```
shape-outside: circle(50% at 50% 50%);
```

La figura fue creada con:

```
border-radius: 50%;
```

Y una imagen para demostrar una manera de uso con:

```
background-image: url('img/profile1.png');
```

Esto permitió mostrar cómo el contenido puede fluir alrededor de un círculo.

7.4.2. Implementación de *ellipse()*

Se implementó una figura ovalada mediante:

```
shape-outside: ellipse(55% 35% at 20% 50%);
```

Esta funcionalidad permite definir radios horizontales y verticales independientes junto a la posición del centro de la elipse dentro del bloque.

7.4.3. Implementación de *inset()*

La propiedad inset() fue implementada para crear rectángulos internos si se requiere bordes redondeados se le aumenta round 20px después del ultimo px:

```
shape-outside: inset(0px 80px 0px 0px);
```

Esta funcionalidad permitió controlar el espacio interno alrededor de los elementos flotantes.

7.4.4. Implementación de *polygon()*

Se desarrolló una figura personalizada mediante coordenadas y una imagen de referencia:

```
shape-outside: polygon(0% 0%, 70% 0%, 75% 40%, 100% 40%,
100% 100%, 0% 100%, 0% 0%);
background-image: url('img/meditar.png');
```

También se utilizó:

```
clip-path: polygon(0% 0%, 70% 0%, 75% 40%, 100% 40%,
100% 100%, 0% 100%, 0% 0%);
```

para que la figura visual coincidiera con el área de flujo del texto.

7.4.5. Implementación de *url()*

Se implementó una forma basada en imágenes utilizando:

```
shape-outside: url("circles.svg");
```

Además, se empleó:

```
shape-image-threshold
```

para controlar el nivel de opacidad utilizado en la detección de la silueta.

7.5. Implementación del Diseño Responsivo

El proyecto fue adaptado para dispositivos móviles mediante Media Queries.

Se implementó:

```
@media (max-width: 720px)
```

Con ello se ajustaron:

- Tamaños de texto
- Distribución de tarjetas
- Espaciados
- Contenedores flexibles

Esto permitió que el sistema funcione correctamente en:

- Computadoras
- Tablets

- Teléfonos móviles

7.6. Uso de Variables CSS

Para mantener coherencia visual se utilizaron variables CSS definidas en :root.

Ejemplo:

```
:root {
    --bg-0: #08111f;
    --bg-1: #0f1d34;
    --bg-2: #17253f;
    --panel: rgba(18, 29, 50, 0.88);
    --panel-strong: rgba(12, 21, 38, 0.95);
    --line: rgba(175, 204, 255, 0.14);
}
```

Las variables facilitaron:

- Reutilización de colores
- Mantenimiento del diseño
- Escalabilidad del proyecto

7.7. Implementación de Efectos Visuales

Se incorporaron distintos efectos para mejorar la experiencia del usuario:

Sombras

box-shadow

Gradientes

linear-gradient()

Transacciones

`transition: all 0.3s ease;`

Efectos Hover

:hover

Estos elementos aportaron dinamismo e interactividad al sistema web

7.8. Pruebas y Validaciones

Durante el desarrollo se realizaron pruebas funcionales y visuales para verificar:

- Correcta visualización de transformaciones
- Funcionamiento de shape-outside
- Compatibilidad con navegadores modernos

- Adaptabilidad responsive
- Correcta alineación de elementos

Las pruebas se realizaron principalmente en:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

7.9. Resultado del Desarrollo

Como resultado final se obtuvo una plataforma web interactiva capaz de enseñar propiedades avanzadas de CSS de forma visual y práctica.

El sistema desarrollado permite:

- Comprender transformaciones CSS
- Visualizar efectos 2D y 3D
- Entender el funcionamiento de shape-outside
- Aprender mediante ejemplos interactivos
- Navegar desde distintos dispositivos

El proyecto cumple con los objetivos planteados inicialmente y demuestra la aplicación práctica de tecnologías modernas de desarrollo web utilizando HTML5 y CSS3.

8. RESULTADOS

Con la realización del presente proyecto se espera obtener una plataforma web interactiva y educativa orientada al aprendizaje visual y práctico de las propiedades avanzadas de CSS, específicamente CSS Transform y CSS Shapes. El sistema permitirá que los usuarios comprendan de manera dinámica el funcionamiento de transformaciones 2D, 3D y flujos de contenido alrededor de formas personalizadas mediante ejemplos interactivos desarrollados en HTML y CSS.

Entre los principales resultados esperados se encuentra la implementación de una interfaz moderna, responsive e intuitiva, capaz de adaptarse a diferentes dispositivos y resoluciones de pantalla. Asimismo, se espera lograr una experiencia de usuario más atractiva gracias al uso de animaciones, efectos visuales, tarjetas interactivas y demostraciones en tiempo real que faciliten la comprensión de conceptos complejos de diseño web.

El proyecto también permitirá demostrar la aplicación práctica de propiedades como `translate()`, `rotate()`, `scale()`, `skew()`, `matrix()`, `shape-outside`, `clip-path` y otras técnicas avanzadas de CSS utilizadas actualmente en el desarrollo frontend moderno. De esta manera, el sistema no solo servirá como material educativo, sino también como una referencia técnica para estudiantes y desarrolladores web interesados en mejorar la presentación visual y la interacción de sus sitios web.

Otro resultado esperado es el fortalecimiento de conocimientos en tecnologías web, aplicando buenas prácticas de desarrollo, organización estructurada del código, diseño responsive y optimización visual. Además, se espera que el proyecto contribuya al aprendizaje autónomo mediante ejemplos claros, documentación explicativa y demostraciones visuales fáciles de entender.

La relevancia del proyecto radica en su aplicabilidad dentro del área de desarrollo web y diseño de interfaces modernas, donde las animaciones y formas personalizadas juegan un papel importante en la experiencia del usuario. Este trabajo podrá ser utilizado como apoyo académico en materias relacionadas con Tecnología Web, Diseño Web y Desarrollo Frontend, así como también servir de base para futuros proyectos educativos o plataformas de aprendizaje interactivo.

9. CONCLUSIONES

A través del desarrollo del presente proyecto se logró investigar e implementar diferentes propiedades avanzadas de CSS relacionadas con transformaciones y formas personalizadas, permitiendo demostrar su utilidad dentro del diseño y desarrollo web moderno. La utilización de tecnologías como HTML y CSS permitió construir una plataforma visual e interactiva capaz de representar de manera práctica conceptos que normalmente resultan complejos en el aprendizaje teórico.

Durante la investigación se identificó que propiedades como `transform`, `shape-outside` y `clip-path` mejoran significativamente la presentación visual y la experiencia del usuario, permitiendo crear interfaces más dinámicas, atractivas y modernas. Asimismo, se comprobó que las transformaciones 2D y 3D facilitan la creación de animaciones e interacciones visuales sin necesidad de utilizar librerías externas o lenguajes adicionales.

También se concluye que el diseño responsive y la correcta organización del código son aspectos fundamentales para garantizar la compatibilidad y mantenibilidad de una aplicación web. La implementación de componentes visuales interactivos permitió fortalecer conocimientos en diseño frontend, estructura semántica y estilos avanzados en CSS.

Finalmente, el proyecto cumplió con los objetivos planteados, proporcionando una guía práctica y educativa que puede servir como material de apoyo para estudiantes y

desarrolladores interesados en profundizar sus conocimientos en tecnologías web modernas.

10. RECOMENDACIONES

Se recomienda continuar ampliando el proyecto incorporando nuevas propiedades y técnicas modernas de CSS, tales como animaciones avanzadas, variables dinámicas, efectos con JavaScript y herramientas de diseño más interactivas. Esto permitiría enriquecer aún más la experiencia de aprendizaje y mejorar la funcionalidad de la plataforma.

Para futuras investigaciones, sería importante integrar frameworks y librerías frontend modernas como Bootstrap, Tailwind CSS o React, con el objetivo de comparar su rendimiento y facilidad de implementación frente al desarrollo tradicional con CSS puro.

Asimismo, se recomienda implementar ejemplos prácticos más complejos y casos reales de aplicación en interfaces modernas, permitiendo que los usuarios comprendan cómo estas propiedades son utilizadas en proyectos profesionales y sitios web actuales.

Finalmente, se sugiere realizar pruebas de usabilidad y accesibilidad para optimizar la experiencia de diferentes tipos de usuarios y garantizar que la plataforma pueda ser utilizada correctamente en distintos dispositivos, navegadores y condiciones de acceso.

11. BIBLIOGRAFIA CITADA

- **W3C. (2018). CSS Shapes Module Level 1. Recuperado de** <https://www.w3.org/TR/css-shapes-1/>
- **Mozilla Developer Network. (2025). CSS transform. Recuperado de** <https://developer.mozilla.org/es/docs/Web/CSS/transform>
- **Mozilla Developer Network. (2025). clip-path. Recuperado de** <https://developer.mozilla.org/es/docs/Web/CSS/clip-path>

Videos Tutoriales

- <https://youtu.be/-dMy4Tw-Igg?list=LL>

Link de Proyectos Desplegados

- https://reymar81.github.io/Shape_INF513/
- https://fabardev.github.io/css_transform/

12. ANEXOS

12.1. Capturas de la interfaz del proyecto

000 SHAPES - DEMOSTRACIÓN VISUAL

Haz que el texto fluya alrededor de formas reales.

SHAPE-OUTSIDE

La base de todo flujo con formas

Esta propiedad solo funciona en elementos flotados. El contorno externo del float define por dónde puede pasar el texto, y ahí es donde entran `circle()`, `ellipse()`, `inset()`, `polygone()`, `url()`, más `shape-margin` y `shape-image-threshold` como complementos de `shape-outside`.

Demostración base sin shape-outside

FORMA NORMAL - SIN SHAPE-OUTSIDE

Este es el estado base: una tarjeta rectangular normal y el texto siguiéndola como cualquier bloque flotado.

Aquí todavía no se está usando ningún tipo de `shape-outside`. El texto choca contra el borde recto del bloque.

01

circle()

Ejemplo con circle()

Usa clásica: una foto de perfil. Con `circle()` el texto fluye de forma natural alrededor del rostro, evitando los huecos vacíos que deja un cuadrado.

En este ejemplo, usamos `img/profile1.png` para demostrar cómo una imagen cuadrada se convierte visualmente en un avatar circular mientras el texto se adapta a su contorno.



```
/* Bloque base en shape_shapes_only.css */
.shape-box {
  float: left;      /* .. habilita shape-outside */
  width: 100px;
  height: 100px;
  margin: 0 10px 12px 0;
}

/* Forma derivada: agrega shape y visual */
.shape-circle {
  shape-outside: circle(50% at 50% 50%); /* .. flujo del texto */
  border-radius: 50%; /* .. aspecto visual */
  background: linear-gradient(...);
}
```

SINTAXIS

`circle(50%)` = radio o tamaño del círculo (puede ser en px, %).

`at 50% 50%` = posición del centro del círculo dentro del contenedor.

`float: left` - sin esto, `shape-outside` no actúa.

05

url()

Ejemplo con url()

La opción más potente: el navegador detecta automáticamente el canal **alfa** (transparencia) de una imagen para crear la forma. Útil para siluetas detalladas.

Cabe destacar que no todos los archivos SVG están preparados para trabajar con `shape-outside: url()`. En algunos casos, las figuras no utilizan `fill="none"` para marcar áreas transparentes, sino que aplican colores con cierta opacidad (por ejemplo, menor a 0.4). Cuando ocurre esto, el navegador interpreta la imagen como un bloque rectangular completo y el párrafo fluye alrededor de ese cuadrado, sin reconocer los detalles internos de la silueta.

Usamos `img/circles.svg`. El texto se ajusta a cada detalle de los círculos sin que tengamos que definir ningún punto manualmente.



```
.shape-url {
  /* Usa la transparencia real del archivo SVG */
  shape-outside: url('img/circles.svg');
  background-image: url('img/circles.svg');
  /* shape-image-threshold: 0.3; */ /*controla qué nivel mínimo de opacidad*/
}
```

SINTAXIS

`fill="black"` = opaco → ese píxel forma parte de la silueta.

`fill="none"` = transparente → el texto puede entrar ahí.

// CSS PROPERTY

transform en profundidad

Mueve, escala, rota e inclina elementos en 2D y 3D. Sin afectar el flujo del documento. Sin mover píxeles reales — solo transformando el sistema de coordenadas.

translate

// FUNCIÓN 2D

translate()

Desplaza el elemento en el eje X y/o Y. El elemento ocupa el mismo espacio en el flujo — solo su imagen se mueve visualmente.

2D No afecta flujo. Resalta px, %, em, rem, vh, vw

```
transform: translate(x, y);
transform: translate(100px); /* solo x y */
transform: translate(100px, -20px); /* x y y */
transform: translate(100%, 0); /* relativo al propio elemento */
```

translateZ() & translate3d()

Trasladan en el eje Z (profundidad). Requieren que el padre tenga perspective para verse el efecto. A más Z positivo → más cerca. Z negativo → más lejos.

3D Necesita perspective en padre. Activa GPU compositing

```
transform: translateZ(100px); /* acerca */
transform: translateZ(-100px); /* aleja */
transform: translate3d(x, y, z);
transform: translate3d(20px, 0, 80px);
```

EDITOR CSS

```
/* El padre ya tiene perspective: 400px */
.elemento {
  transform: translateZ(40);

  width: 70px;
  height: 70px;
  background: #4af500;
  border-radius: 8px;
}
```

CDS
Enter
para copiar

PREVIEW

scale()

Escala el elemento desde su transform-origin. Un valor de 1 es tamaño original, 2 = doble, 0.5 = mitad. Valores negativos reflejan el elemento.

2D Valores negativos reflejan. Sin unidades (factor)

```
transform: scale(factor); /* igual en x y y */
transform: scale(1.5); /* 150% */
transform: scale(0.5, 2); /* x = 50%, y = 200% */
transform: scale(-1, 1); /* escala horizontal */
```

EDITOR CSS

```
/* Prueba scale(-1, 1) para espejo */
.elemento {
  transform: scale(-1, 0);

  width: 60px;
  height: 60px;
  background: #ff6600;
  border-radius: 8px;
}
```

CDS
Enter
para copiar

PREVIEW

VARIANTES

scaleX(ex) solo horizontal

scaleY(ey) solo vertical

scaleX / scaleY

Escala/comprime solo en el eje X. Escala/comprime solo en el eje Y. Escala en el eje Z. Solo visible.

12.2. Estructura del Proyecto

